

# Como os Computadores Realmente Funcionam

## Estudo Aprofundado: Do Elétron ao Algoritmo

DeepStudy PDF Generator

July 31, 2026

# Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Fundamentos e Teoria Essencial</b>                    | <b>2</b>  |
| 1.1      | Transistores e portas lógicas . . . . .                  | 2         |
| 1.2      | Representação binária e sistemas de numeração . . . . .  | 3         |
| 1.3      | Arquitetura von Neumann . . . . .                        | 3         |
| <b>2</b> | <b>Por Debaixo dos Panos</b>                             | <b>4</b>  |
| 2.1      | Os blocos e os barramentos . . . . .                     | 4         |
| 2.2      | O ciclo fetch-decode-execute . . . . .                   | 5         |
| 2.3      | A hierarquia de memória . . . . .                        | 5         |
| <b>3</b> | <b>Implementação e Exemplos</b>                          | <b>7</b>  |
| 3.1      | Como um programa vira instruções de máquina . . . . .    | 7         |
| 3.2      | Um exemplo de ponta a ponta: somar um array . . . . .    | 7         |
| 3.3      | Análise de desempenho . . . . .                          | 8         |
| <b>4</b> | <b>Tópicos Avançados</b>                                 | <b>10</b> |
| 4.1      | Quando o pipeline quebra: desvios condicionais . . . . . | 10        |
| 4.2      | Cache miss em produção: um caso real . . . . .           | 10        |
| 4.3      | Debugging: quando o compilador "mente" . . . . .         | 11        |
| <b>A</b> | <b>Referências e Leitura Adicional</b>                   | <b>12</b> |
| <b>B</b> | <b>Glossário</b>                                         | <b>13</b> |
|          | <b>Index</b>                                             | <b>15</b> |

# Chapter 1

## Fundamentos e Teoria Essencial

Um computador moderno é, na base, uma pilha de abstrações: algoritmos viram código-fonte, código-fonte vira instruções de máquina, instruções de máquina viram sinais elétricos, e sinais elétricos são apenas voltagem sendo ligada e desligada em bilhões de **transistores** por segundo. Este capítulo começa no fundo dessa pilha — o elétron — e sobe até o ponto onde um programa em uma linguagem de alto nível passa a fazer sentido para o hardware.

### 1.1 Transistores e portas lógicas

Um **transistor** é, para fins práticos, uma chave elétrica controlada por eletricidade: aplicando voltagem em um terminal (a *porta*, ou *gate*), ele permite ou bloqueia a passagem de corrente entre outros dois terminais. Não há nada "inteligente" em um transistor isolado — a inteligência emerge de como bilhões deles são conectados.

Combinando poucos transistores é possível construir **portas lógicas**: circuitos que implementam uma função booleana. A Figura 1.1 mostra a porta mais fundamental para a eletrônica digital moderna, a **NAND**, e sua tabela-verdade — toda porta lógica (AND, OR, NOT, XOR...) pode ser construída combinando apenas portas NAND, e é por isso que ela é chamada de porta *funcionalmente completa*.

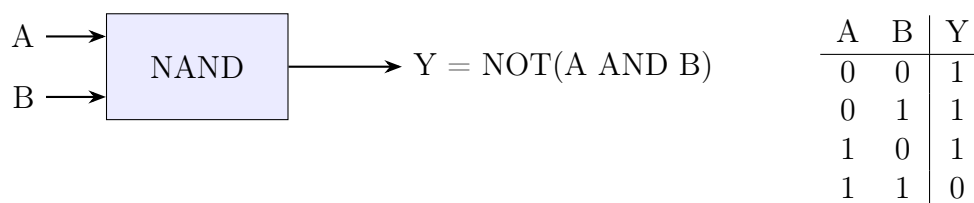


Figure 1.1: Porta NAND e sua tabela-verdade. É a única porta que a maioria dos processadores usa como bloco de construção físico.

Um exemplo concreto de por que isso importa: um **somador de 1 bit** (*half adder*) — o circuito que soma dois bits e produz uma soma e um "vai um" (*carry*) — é construído com uma porta XOR (soma) e uma porta AND (carry). Encadear 32 ou 64 desses somadores,

com ajustes para propagar o carry, é literalmente o que existe dentro da ALU (Unidade Lógica e Aritmética) de uma CPU quando ela executa uma instrução de soma inteira.

## 1.2 Representação binária e sistemas de numeração

Um transistor tem, no fundo, apenas dois estados estáveis relevantes: "conduzindo corrente" e "não conduzindo". Por isso toda informação em um computador — números, texto, imagens, instruções — é codificada em **binário** (base 2), onde cada dígito (*bit*) vale 0 ou 1.

Um número em binário é lido como qualquer sistema posicional, trocando potências de 10 por potências de 2:

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11_{10}$$

Na prática, programadores raramente escrevem binário puro — usam **hexadecimal** (base 16) como atalho, porque cada dígito hexadecimal representa exatamente 4 bits (um *nibble*), tornando a conversão trivial de cabeça:

$$0xB = 1011_2, \quad 0x2F = 0010\,1111_2$$

Números negativos são representados quase universalmente em **complemento de dois**: para negar um número, inverte-se todos os bits e soma-se 1. A vantagem que fez esse esquema vencer é que o hardware pode usar o mesmo circuito somador tanto para soma quanto para subtração — subtrair é apenas somar o complemento de dois do segundo operando.

## 1.3 Arquitetura von Neumann

Quase todo processador de propósito geral hoje segue, em maior ou menor grau, a **arquitetura von Neumann**, descrita em 1945 por John von Neumann. A ideia central — revolucionária na época — é que *instruções e dados compartilham a mesma memória*. Antes disso, máquinas como o ENIAC eram "programadas" fisicamente, reconectando cabos e chaves; von Neumann propôs armazenar o próprio programa como dados na memória, o que significa que um computador pode modificar, gerar ou carregar programas em tempo de execução — é a base de tudo, de compiladores a máquinas virtuais.

O detalhe explorado no Capítulo 2 é justamente essa decisão: como CPU e memória se conectam para tornar isso possível, e qual o preço que se paga por compartilhar um único caminho entre instrução e dado (o chamado *gargalo de von Neumann*).

# Chapter 2

## Por Debaixo dos Panos

### 2.1 Os blocos e os barramentos

A Figura 2.1 mostra os blocos de uma máquina von Neumann e como eles se conectam. A CPU não tem um único fio para a memória — tem três **barramentos** distintos, cada um com um papel:

- **Barramento de endereço** — unidirecional, sai da CPU. Carrega *qual* posição de memória a CPU quer ler ou escrever. A largura desse barramento define o espaço de endereçamento: 32 linhas endereçam  $2^{32}$  posições (4 GB).
- **Barramento de dados** — bidirecional. Carrega o *conteúdo* de fato: a CPU lê dados dele (memória → CPU) ou escreve dados nele (CPU → memória).
- **Barramento de controle** — carrega sinais de coordenação: "isto é uma leitura ou uma escrita?", "o dado já está pronto no barramento?", interrupções, clock.

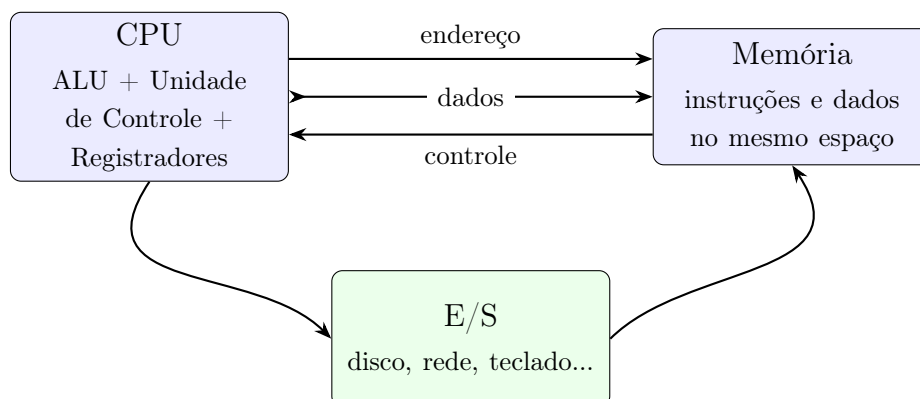


Figure 2.1: Arquitetura von Neumann: CPU e memória compartilham um único espaço de endereçamento, conectados por três barramentos com papéis diferentes.

## 2.2 O ciclo fetch-decode-execute

Tudo o que uma CPU faz, o tempo todo, é repetir um ciclo de três (ou quatro, contando a escrita do resultado) etapas — o **ciclo fetch-decode-execute** (Figura 2.2):

1. **Fetch (busca)**: a Unidade de Controle lê o endereço guardado no registrador **PC** (*Program Counter*) e busca a instrução armazenada naquele endereço da memória, colocando-a no **registrador de instrução**. O PC é então incrementado para apontar para a próxima instrução.
2. **Decode (decodificação)**: o hardware interpreta os bits da instrução para descobrir qual operação executar e quais registradores/valores ela usa como operandos.
3. **Execute (execução)**: a ALU realiza a operação (soma, comparação, deslocamento de bits...) ou, no caso de uma instrução de memória, a CPU calcula o endereço efetivo a ser lido/escrito.
4. **Writeback (escrita do resultado)**: o resultado é gravado de volta em um registrador ou em memória.

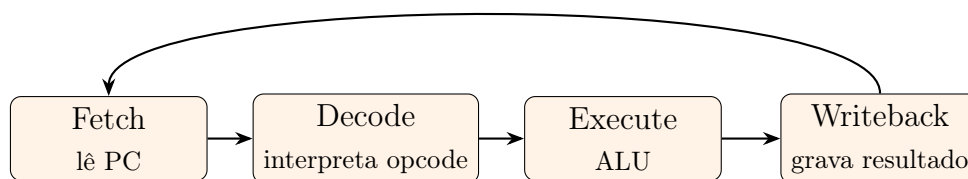


Figure 2.2: O ciclo fetch-decode-execute: cada instrução de um programa passa por essas quatro etapas antes que a próxima comece.

Processadores modernos *sobrepõem* essas etapas via **pipelining** — enquanto uma instrução está na etapa de execute, a próxima já está sendo decodificada, e uma terceira já está sendo buscada. Isso não reduz a latência de uma instrução individual, mas aumenta drasticamente o *throughput* (instruções completadas por segundo). O Capítulo 4 volta a esse ponto ao discutir o que acontece quando o pipeline é interrompido por um desvio condicional mal previsto.

## 2.3 A hierarquia de memória

Existe uma tensão física fundamental: memória rápida é cara e tem pouca capacidade; memória barata e de alta capacidade é lenta. A solução da indústria é a **hierarquia de memória** (Figura 2.3) — camadas cada vez maiores e mais lentas entre a CPU e o armazenamento permanente:

O motivo pelo qual isso funciona na prática — em vez de a CPU passar o tempo todo esperando a RAM — é o **princípio da localidade**:

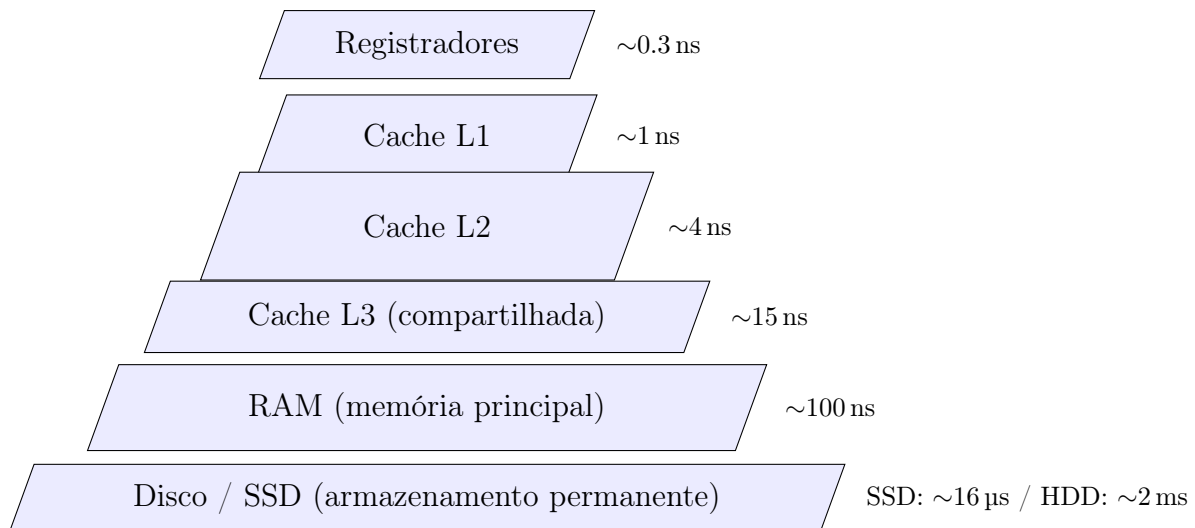


Figure 2.3: Hierarquia de memória, do mais rápido/menor (topo) ao mais lento/maior (base). Ordens de grandeza de latência típicas em hardware de servidor comum.

- *Localidade temporal*: se um endereço foi acessado agora, é provável que seja acessado de novo em breve (ex.: variável de um laço).
- *Localidade espacial*: se um endereço foi acessado, é provável que endereços vizinhos sejam acessados em seguida (ex.: elementos consecutivos de um array).

Por isso as caches não trazem um byte por vez: trazem **linhas de cache** inteiras (tipicamente 64 bytes) de uma vez, apostando que os bytes vizinhos serão usados em breve — o que explica por que percorrer um array sequencialmente é ordens de magnitude mais rápido do que percorrê-lo em ordem aleatória, mesmo fazendo exatamente o mesmo número de leituras.

# Chapter 3

## Implementação e Exemplos

### 3.1 Como um programa vira instruções de máquina

O caminho de um arquivo de código-fonte até bits sendo executados pela CPU passa por vários estágios de tradução (Figura 3.1). Para uma linguagem compilada tradicional:

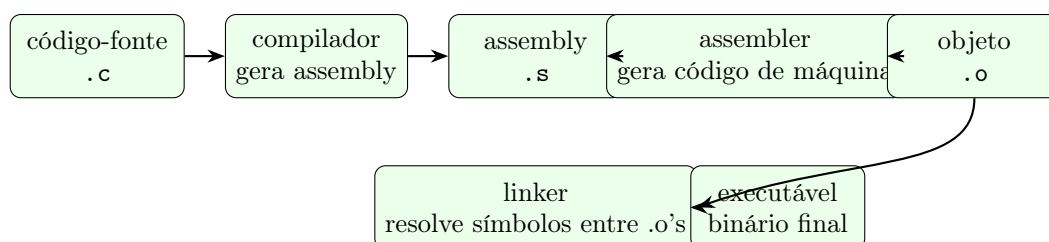


Figure 3.1: Pipeline de tradução: código-fonte → assembly → código-objeto → executável ligado.

#### E o TypeScript?

TypeScript segue um caminho diferente: o `tsc` não gera assembly nem código de máquina — ele apenas *apaga os tipos* e transpila para JavaScript comum. Quem de fato vira código de máquina é o JavaScript resultante, e isso acontece em tempo de execução dentro de um motor como o V8: o código é primeiro interpretado, e os trechos "quentes" (executados repetidamente) são compilados *just-in-time* (JIT) para código de máquina nativo, com otimizações que podem ser desfeitas (*deoptimization*) se as suposições feitas deixarem de valer.

### 3.2 Um exemplo de ponta a ponta: somar um array

Para tornar isso concreto, eis a mesma operação — somar os elementos de um array — em três níveis de abstração diferentes.

**Nível alto (TypeScript):** o que um programador escreve e lê.

```
function somaArray(valores: number[]): number {
  let acumulador = 0;
  for (let indice = 0; indice < valores.length; indice++) {
    acumulador += valores[indice];
  }
  return acumulador;
}
```

**Nível baixo (assembly RISC-V, RV32I):** o que um compilador C equivalente geraria — a sintaxe é deliberadamente minimalista (*reduced instruction set computer*), com poucas instruções, todas de largura fixa:

```
# soma_array(int* a0, int a1) -> resultado em a0
# a0 = endereco base do array, a1 = numero de elementos
soma_array:
    li    t0, 0          # acumulador = 0
    li    t1, 0          # indice = 0
loop:
    beq    t1, a1, fim    # se indice == n, pula para fim
    slli   t2, t1, 2      # t2 = indice * 4 (int = 4 bytes)
    add    t2, a0, t2     # t2 = endereco do elemento
    lw     t3, 0(t2)      # t3 = array[indice] (le da memoria)
    add    t0, t0, t3     # acumulador += array[indice]
    addi   t1, t1, 1      # indice++
    j      loop
fim:
    mv     a0, t0         # valor de retorno vai em a0
    ret
```

Cada linha de assembly corresponde, quase 1 para 1, a um passo do ciclo fetch-decode-execute descrito no Capítulo 2: a instrução `lw` (*load word*) é a única linha que de fato usa o barramento de dados para ler da memória — todo o resto do laço opera inteiramente dentro dos registradores da CPU, que é ordens de magnitude mais rápido, exatamente pela hierarquia de memória discutida no Capítulo 2.

**Nível mais baixo (opcodes):** cada instrução RISC-V acima é, na memória, apenas 32 bits fixos. A instrução `addi t1, t1, 1` (RV32I), por exemplo, é codificada como um valor decimal como `0x00130313` — os bits mais significativos identificam o *opcode* (que a Unidade de Controle decodifica como "operação imediata"), e os demais campos identificam os registradores de origem/destino e a constante 1. É literalmente esse padrão de bits, e não texto, que trafega pelo barramento de dados até o registrador de instrução da CPU.

### 3.3 Análise de desempenho

A diferença entre acessar memória sequencialmente e aleatoriamente, prevista pela hierarquia de memória do Capítulo 2, é mensurável, não apenas teórica. Somar 64 MB de

inteiros sequencialmente costuma rodar várias vezes mais rápido que somar os mesmos 64 MB em ordem embaralhada, no mesmo hardware, com o mesmo número de operações — a diferença inteira vem de quantas dessas leituras acertam a cache (*cache hit*) versus obrigam a esperar a RAM (*cache miss*). Escrever algoritmos "cache-friendly" — percorrer arrays na ordem em que estão na memória, evitar ponteiros espalhados — costuma valer mais para desempenho do que trocar de algoritmo com melhor complexidade assintótica em conjuntos de dados de tamanho moderado.

# Chapter 4

## Tópicos Avançados

### 4.1 Quando o pipeline quebra: desvios condicionais

O Capítulo 2 descreveu o pipelining como sobrepor fetch/decode/execute de instruções consecutivas. Isso funciona bem quando a CPU sabe, de antemão, qual é a próxima instrução a buscar. Mas o exemplo de assembly do Capítulo 3 tem uma instrução `beq` (*branch if equal*) — um desvio condicional — e nesse caso a CPU literalmente não sabe qual instrução vem a seguir até o *execute* do `beq` terminar, que é tarde demais para já ter buscado e decodificado a próxima.

A solução da indústria é **predição de desvio** (*branch prediction*): a CPU aposta, com base no histórico daquele desvio específico, se ele vai ser tomado ou não, e já começa a buscar/decodificar/executar especulativamente as instruções desse caminho. Se a previsão está certa, ganhou-se tempo; se está errada, o pipeline inteiro daquelas instruções especulativas precisa ser descartado (*pipeline flush*) — um custo real, tipicamente de 10 a 20 ciclos de clock perdidos.

Isso tem uma consequência prática direta para quem escreve código de alto nível: laços com um padrão de desvio previsível (ex.: `if (par)` alternando de forma regular) são mais lentos para o preditor de desvio "aprender" do que laços com um padrão sempre igual (sempre verdadeiro, ou sempre falso) ou totalmente aleatório (onde a CPU nem tenta prever de forma sofisticada). Código com muitos desvios imprevisíveis dentro de um laço quente pode ficar sensivelmente mais lento que uma versão logicamente equivalente sem desvios — por exemplo, substituindo um `if` por uma operação aritmética/bitwise sem desvio, técnica conhecida como **branchless programming**.

### 4.2 Cache miss em produção: um caso real

Um sintoma clássico em sistemas de produção: duas implementações com a *mesma* complexidade assintótica ( $O(n)$ ) e o mesmo número de operações, mas uma consistentemente mais lenta. A causa mais comum não é o algoritmo — é o padrão de acesso à memória.

Percorrer uma matriz grande por colunas (quando ela está armazenada *por linhas*, como

é o padrão em quase toda linguagem) força uma falha de cache (*cache miss*) a cada elemento, porque cada acesso salta o tamanho de uma linha inteira na memória — muito além dos 64 bytes que uma linha de cache traz de uma vez (Capítulo 2). Trocar a ordem dos laços para percorrer por linhas resolve o problema sem mudar uma única operação lógica do algoritmo. Isso é o motivo pelo qual "otimizar" às vezes significa reorganizar o *acesso* aos dados, não a lógica.

## 4.3 Debugging: quando o compilador "mente"

Ao depurar código otimizado (build de release), é comum uma variável "desaparecer" no debugger ou a execução "pular" linhas — não é um bug do debugger, é o compilador tendo eliminado a variável (porque ela cabia inteira em um registrador e nunca foi escrita de volta à memória) ou reordenado instruções que não tinham dependência de dados entre si. Isso é diretamente ligado ao ciclo fetch-decode-execute e à hierarquia de registradores/memória: o compilador só reordena ou elimina o que sabe, pela análise do código, que não muda o resultado observável — mas "observável" nem sempre bate com a intuição de quem está lendo o código-fonte linha a linha em um debugger.

# Appendix A

## Referências e Leitura Adicional

- Patterson, D. A.; Hennessy, J. L. *Computer Organization and Design: The Hardware/Software Interface* — a referência clássica para arquitetura de computadores, usa MIPS/RISC-V nas edições recentes.
- RISC-V International. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*: <https://riscv.org/technical/specifications/>
- Documentação oficial do TypeScript, seção sobre o compilador `tsc` e emissão de JavaScript: <https://www.typescriptlang.org/docs/handbook/compiler-options.html>
- V8 Team. "A tour of V8: Garbage Collection" e artigos relacionados sobre o pipeline de interpretação/JIT do V8: <https://v8.dev/blog>
- Números de latência aproximados de hardware moderno ("Latency Numbers Every Programmer Should Know"), popularizados por Jeff Dean (Google) e mantidos atualizados pela comunidade: [https://colin-scott.github.io/personal\\_website/research/interactive\\_latency.html](https://colin-scott.github.io/personal_website/research/interactive_latency.html)

# Appendix B

## Glossário

**Transistor:** chave elétrica controlada por voltagem; bloco de construção físico de toda porta lógica.

**Porta lógica:** circuito que implementa uma função booleana (AND, OR, NOT, NAND, XOR...) combinando transistores.

**Binário:** sistema de numeração posicional em base 2, a representação nativa de qualquer valor em hardware digital.

**Complemento de dois:** esquema padrão para representar números negativos em binário, que permite reusar o circuito de soma para fazer subtração.

**Arquitetura von Neumann:** modelo onde instruções e dados compartilham o mesmo espaço de memória e o mesmo barramento.

**Barramento:** conjunto de fios/trilhas que transportam sinais entre CPU, memória e periféricos — de endereço, de dados ou de controle.

**Ciclo fetch-decode-execute:** sequência repetida pela CPU para processar cada instrução: buscar, decodificar, executar e gravar o resultado.

**Program Counter (PC):** registrador que guarda o endereço da próxima instrução a ser buscada.

**Pipelining:** sobreposição das etapas de fetch/decode/execute de instruções consecutivas para aumentar o throughput da CPU.

**Hierarquia de memória:** camadas de memória com trade-off crescente de capacidade por velocidade — registradores, caches L1/L2/L3, RAM, disco.

**Linha de cache:** menor unidade de dados movida entre a memória principal e a cache, tipicamente 64 bytes.

**RISC-V:** arquitetura de conjunto de instruções (ISA) aberta e de código livre, com um conjunto de instruções deliberadamente pequeno e uniforme.

**Predição de desvio:** técnica de hardware para adivinhar o resultado de um desvio condicional antes de ele ser de fato resolvido, mantendo o pipeline ocupado.

**Branchless programming:** técnica de escrever lógica condicional sem instruções de desvio, evitando o custo de previsões erradas.

# Index

arquitetura von Neumann, 3, 13

barramento, 4, 13

    de controle, 4

    de dados, 4

    de endereço, 4

binário, 3, 13

branchless programming, 10, 14

ciclo fetch-decode-execute, 5, 13

complemento de dois, 3, 13

hexadecimal, 3

hierarquia de memória, 5, 13

linha de cache, 6, 13

pipelining, 5, 13

porta lógica, 2, 13

predição de desvio, 10, 13

program counter, 5, 13

RISC-V, 8, 13

transistor, 2, 13